

Week 7 Writeup - Develop Second modeling approach

Langley, Jack, Chan, Ryan

Github Repo: [jmetz1313/spotify-project](https://github.com/jmetz1313/spotify-project)

Written Report: the report should address the following questions/topics:

- The model approach you chose to build for the week and why that makes sense for the prediction task
- Discuss the complexity of the modeling approach
- Hyperparameters evaluated and why you chose to evaluate them
- Identify model performance metrics and discuss why those metrics make sense for the model you chose and your prediction task
- Calculate the metrics using training and validation datasets
- For all 3 variations calculate the metrics for both training and validation datasets
- Discuss and analyze how training and validation metrics change across variations
- Compare performance metrics for the validation dataset across all 3 variations
- Provide a table that shows 3 variations, training and validation values for all metrics you chose
- Identify the best model for the week and discuss why you selected that model
- Submission requirements: 10-page max, 1.5-spaced paragraphs, 11-sized font, a professional font format. An unlimited number of appendix pages. Submitted as a google doc (link - make sure you share your doc with the instructor)

Embeddings

To improve the caliber of our recommendation system, we used precomputed user and tweet embeddings in this week's modeling method. We converted the vector files into Twitter embeddings, which could increase the model's complexity and its ability to make determinations. These embeddings served as structured input for our KNN model, which is useful for enhancing the model's accuracy.

This embedding consists of the raw and sparse model. Embeddings transformed these inputs into dense vectors that captured intricate associations, making them perfect for use with similarity-based techniques like KNN. To get the best tweet recommendations for each user, we specifically employed the cosine similarity between the user and tweet embedding vectors.

Using cosine similarity as the matching measure, a similarity matrix was built using user embeddings as rows and tweet embeddings as columns to get the top-N relevant tweets for each user. Our effective approach is to reduce the dimensionality by using raw data embeddings, which helps prevent overfitting and allows for the use of a weighted average. During similarity assessment, embeddings used bespoke weights to prioritize recent or highly engaged tweets.

We tracked the trade-off between suggestion accuracy and computing efficiency across these modifications. Weighted embeddings marginally increased relevance for trending content, whereas reduced-dimensional embeddings, as predicted, produced faster calculations with relatively little performance impact.

Scoring/Accuracy Method:

Due to the nature of our model, the output is a song recommendation rather than a traditional classification or numeric prediction. As a result, standard accuracy metrics did not directly apply. To evaluate the quality and relevance of each of our model's recommendations, our group manually scored the outputs for a fixed testing sample of 100 songs, allowing for both a consistent and comparable evaluation of each model.

Test set song recommendations were evaluated on a binary scoring system: a score of 1 represents a relevant/strong song recommendation, while a 0 represents an irrelevant/poor song recommendation. Although we acknowledge this scoring system is imperfect and subjective, the best effort to score recommendations on lyrical content and overall "vibe" (i.e. a listener who enjoyed the input song might reasonably enjoy the recommended one). It is important to reiterate accuracy scoring was based on perceived song similarity and recommendation quality, not based on personal music preference. Our test set contained a good variety of randomly selected songs, covering a number of genres and eras.

Given the subjective nature of grading a single recommendation as good or bad, we have added a $\pm 5\%$ error to the graded score for each model. We felt in a 100 song sample, we were able to accurately grade positive or negative within 5 songs of error.

Given a longer project timeline and additional resources, we could have evaluated model accuracy using real-time user feedback to better capture preferences and listener behavior. This would have allowed us to incorporate a feedback/supervised training aspect in our model. However, due to the limited nature of this course and project, we chose an evaluation method that was both simple and immediately accessible. This allows us to develop, test, and compare multiple versions of the model while maintaining a consistent accuracy metric.

The test set contained songs we were both familiar and unfamiliar with. It is estimated that each song (input and recommended) was listened to for a minimum of one minute (longer for unfamiliar songs) with the lyrics on screen before making a scoring decision. Each round of scoring (done three times) is estimated to take approximately three hours.

New Model 2: Twitter Embedding

This new model we used this week was chosen to compare the efficacy of a new embedding against the one we used in last week's model. From this point, we could see which one gave more accurate recommendations and proceed to add our numerical features to the better of the two embedding models. We ended up proceeding with this new twitter embedding model because it was able to slightly outperform that of last week.

New Model 3: Twitter Lyric Embeddings + Numerical Feature Integration

Our second new modeling approach for the week worked to incorporate the other numerical features of the dataset beyond lyric embeddings. In this model the one-hot encoded genre and subgenre columns were omitted. Feature engineering and normalization was completed in week 5. The new variables include: **popularity**, **square root of BPM**, **dance**, **square root of acoustic**, **happy**, **power** (*loud X energy*), **camelot_sin**, and **camelot_cos**.

This model introduced a new layer of complexity compared to the Week 6 model, which relied solely on lyric embeddings and did not incorporate any musical or structural features. Our

hypothesis was that combining lyrics with musical metadata would yield more holistic recommendations - matching not only the song's message but also its tempo and overall vibe.

The integration of the new features was straightforward, as it followed the same processing pipeline as our previous two embedding-based models. Due to the time-intensive nature of the manual accuracy scoring, we limited the combined feature and embedding model to the word2vec (Twitter) embeddings, which achieved a slightly better accuracy metric than GloVe. Additionally, we felt that word2vec, trained on informal Twitter language, aligned better with the structure and style of song lyrics and their informal nature.

Training time remained comparable to earlier models. The manual accuracy score for this model was 49 ± 5 , reflecting a 44–54% relevant recommendation accuracy range after accounting for subjectivity in scoring.

NEW MODEL: ADDING ALL NUMERICAL FEATURES USING THE TWITTER (WORD2VEC) EMBEDDINGS. EXCLUDES GENRE AND SUBGENRE DATA.

```
In [46]: # Get list of metadata features (drop genre/subgenre/text columns)
columns_to_exclude = [col for col in df_train_encoded.columns
                      if col.startswith('Genre_') or col.startswith('Subgenre_') or
                      col in ['Song', 'Artist', 'Clean_Lyrics', 'Tokens', 'Embeddings', 'Twitter_Embeddings']]

# Select numeric metadata features
X_train_meta = df_train_encoded.drop(columns=columns_to_exclude).select_dtypes(include=[np.number])
X_test_meta = df_test_encoded.drop(columns=columns_to_exclude).select_dtypes(include=[np.number])

# Combine metadata + Embeddings
X_train_full = np.hstack([X_train_meta.values, np.stack(df_train_encoded['Embeddings'].values)])
X_test_full = np.hstack([X_test_meta.values, np.stack(df_test_encoded['Embeddings'].values)])

# Fit the KNN_FULL model
KNN_FULL = NearestNeighbors(n_neighbors=5, metric='cosine')
KNN_FULL.fit(X_train_full)

# Example: Recommend songs for a test song
test_song_idx = 10
distances, indices = KNN_FULL.kneighbors([X_test_full[test_song_idx]])

print(f"Recommendations for test song: {df_test_encoded['Song'].iloc[test_song_idx]} by {df_test_encoded['Artist'].iloc[test_song_idx]}")
for rank, (idx, dist) in enumerate(zip(indices[0], distances[0]), 1):
    song_name = df_train_encoded['Song'].iloc[idx]
    artist_name = df_train_encoded['Artist'].iloc[idx]
    print(f"{rank}. {song_name} by {artist_name} (distance: {dist:.4f})")
```

Figure 1: Screenshot of the code incorporating the new numerical features into the model.

Reason for the HyperParameter

Our group tackled many questions regarding hyperparameter tuning of K for our KNN model last week, and while we have made adjustments to the ways we are adding features and embedding the lyrics vector this week, nothing material has changed the way we are approaching hyperparameter tuning.

As our model is not one that relies on traditional accuracy measurements, we don't see the utility in working to maximize our expected accuracy based on different values of K. We care more about the actual order of distance between our song and the recommended songs rather than the physical distance between the points. Changing our K only changes the distance and does nothing to the order at which the recommendations are provided.

We plan to address more hyperparameters in our model next week when we begin to tackle things like feature weighting for genres and other things, we will see an uptick in this section and the importance of handling hyperparameters, but for this week we reverberate the same sentiment we shared last week.

Performance Comparison (1 page)

We saw tangible improvements in recommendation accuracy, gaining 6% accuracy from our previous best model. Nearing 50% accuracy is important for us, as this is still a very simple model with no added weights, no genre inputs, and no further tuning. We believe that there is still much more room for improvement in the coming weeks to expand upon this, but are happy with where we stand in this process.

While these measurements are subjective, the fact that our new score of 49% exceeds our hypothetical confidence interval of the previous model suggests that we are fairly confident that adding the numerical features did improve the quality of our model. The table below shows our models produced thus far and the improvements in accuracy.

Model	Accuracy	Confidence Interval	K Parameter
KNN	41/100	36 to 46	1
KNN_Twitter	43/100	38 to 48	1
KNN_Full	49/100	44 to 54	1

Table 1: Comparison of the three model performances with ± 5 confidence intervals. KNN_Full showed the best performance.